

CHAPTER · ENSEMBLE III

Gradient Boosting 의 후예들 XGBoost · LightGBM · CatBoost

같은 뼈대 (잔차를 트리로 반복 적합) 위에 , 각자 무엇을 더했나 ? 정규화 · 속도 · 범주형 변수

Boosting 강의 다음 시간 · 튜닝은 별도 슬라이드 (Boosting 모형 튜닝) 에서 다룹니다

학습 목표

1 왜 새 구현이 필요했나

고전적 GBM 의 한계 (느린 분할 탐색 , 정규화 부재 , 범주형 · 결측 처리) 를 이해한다 .

2 XGBoost

정규화된 목적함수 , 2 차 근사 (뉴턴 단계) , 결측치 기본 방향을 설명할 수 있다 .

3 LightGBM

히스토그램 분할 , leaf-wise 성장 , GOSS·EFB 로 빨라지는 원리를 이해한다 .

4 CatBoost 와 비교

Ordered target statistics, ordered boosting, 대칭 트리를 이해하고 세 모델을 비교해 선택할 수 있다 .

복습 : Gradient Boosting 과 그 한계

$$f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \nu \cdot h_m(\mathbf{x}), \quad h_m = \text{tree}(r_i \sim \mathbf{x}), \quad r_i = -\partial L / \partial f(\mathbf{x}_i)$$

느린 분할 탐색

매 분할마다 모든 변수를 정렬하고 모든 분할점을 검사 → 데이터가 커지면 매우 느림

명시적 정규화 부재

트리 복잡도에 대한 벌점이 목적함수에 없음 → ν , 깊이 등으로만 간접 조절

1 차 정보만 사용

기울기 (방향) 만 쓰고 곡률 (2 차 미분) 은 활용하지 않음

결측치 · 범주형 처리

미리 채우거나 원 - 핫 인코딩 필요 . 수준이 많은 범주형 변수에 약함

세 후예는 모두 이 한계들을 각자의 방식으로 해결한 “같은 알고리즘의 개선판” 이다

가계도 : 누가, 언제, 무엇을 더했나

세 라이브러리 모두 지금도 활발히 개발 중이며 R · Python 에서 사용 가능

2001

2014 · 2016

2017

2017 · 2018

Gradient Boosting

J. Friedman

잔차 (음의 기울기) 를 트리로
반복 적합

XGBoost

T. Chen & C. Guestrin (워싱턴대)

정규화 + 2 차 근사 + 시스템
최적화 . Kaggle 을 휩쓸며
대중화

LightGBM

Microsoft

히스토그램 + leaf-wise +
GOSS·EFB → 대용량에서
빠르게

CatBoost

Yandex

범주형 변수의 ordered
target statistics + ordered
boosting

PART 1

XGBoost

eXtreme Gradient Boosting — “정규화 + 2 차 근사 + 빠른 시스템”

XGBoost 의 세 기둥

1

정규화된 목적함수

손실 + 트리 복잡도 벌점 (잎의 수, 잎 값의 크기). 과대적합을 목적함수 차원에서 제어

2

2 차 근사 (뉴턴 단계)

손실의 1 차 미분 (g) 과 2 차 미분 (h) 을 함께 사용 → 잎의 최적값과 분할 이득을 닫힌 식으로 계산

3

시스템 최적화

결측치 기본 방향 학습, 근사 / 히스토그램 분할, 병렬 처리, 캐시 최적화, GPU·분산 학습

한마디로 : Friedman 의 gradient boosting 을 “통계적으로 더 엄밀하게 (정규화 · 뉴턴 단계), 공학적으로 훨씬 빠르게” 만든 구현 . 2014~2016 년 Kaggle 우승 솔루션의 표준이 되었다

정규화된 목적함수

$$\text{Obj} = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k), \quad \Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_j w_j^2$$

$l(y, \hat{y})$

손실 (적합도)

제곱오차, 로지스틱 등 — 기존 GBM 과 동일

γT

잎의 개수 T 에 대한 벌점

잎 하나를 늘리려면 손실이 γ 이상 줄어야 함
→ 가지치기 효과

$\frac{1}{2}\lambda \sum w^2$

잎 값 w_j 의 크기에 대한 벌점 (L2)

잎 값을 0 쪽으로 줄임 (shrink) → 극단적 예측 방지 . $\propto \sum |w|$ (L1) 도 가능

릿지 회귀 ($\lambda \Sigma \beta^2$) 와 같은 발상을 트리의 잎 값에 적용한 것 . 파라미터 이름: gamma, lambda, alpha

2 차 근사 : 앞의 최적값을 닫힌 식으로

각 관측치의 1 차·2 차 미분 : $g_i = \partial l / \partial \hat{y}$, $h_i = \partial^2 l / \partial \hat{y}^2$, 앞 j 에 모인 관측치들의 합 : $G_j = \sum g_i$, $H_j = \sum h_i$

앞 j 의 최적값

$$w_j^* = -G_j / (H_j + \lambda)$$

그때의 목적함수 값 (트리 구조의 점수)

$$Obj^* = -1/2 \sum_j G_j^2 / (H_j + \lambda) + \gamma T$$

제곱오차일 때 확인해 보면

$$l = 1/2(y - \hat{y})^2 \rightarrow g_i = \hat{y}_i - y_i = -(\text{잔차}), \quad h_i = 1 \Rightarrow w_j^* = \sum(\text{잔차}) / (n_j + \lambda)$$

= 앞 안 잔차의 평균을 0 쪽으로 조금 줄인 값 . $\lambda = 0$ 이면 지난 시간의 “잔차를 트리로 적합” 과 정확히 같다

분류 (로지스틱) 에서는 $h_i = \hat{p}_i(1 - \hat{p}_i)$: 확신이 없는 ($\hat{p} \approx 0.5$) 관측치일수록 곡률이 크다 $\rightarrow$ 뉴턴 방법처럼 곡률로 보폭을 자동 조절

분할 이득 (Gain) 과 γ

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{(H_L + \lambda)} + \frac{G_R^2}{(H_R + \lambda)} - \frac{(G_L + G_R)^2}{(H_L + H_R + \lambda)} \right] - \gamma$$

왼쪽 자식 점수 + 오른쪽 자식 점수 - 나누기 전 점수 - 앞 하나 늘리는 비용

분할 후보 비교

모든 변수 · 분할점 후보의 Gain 을 계산해
가장 큰 것을 선택 (지난 시간 RSS 감소와
같은 역할)

γ (gamma, min_split_loss)

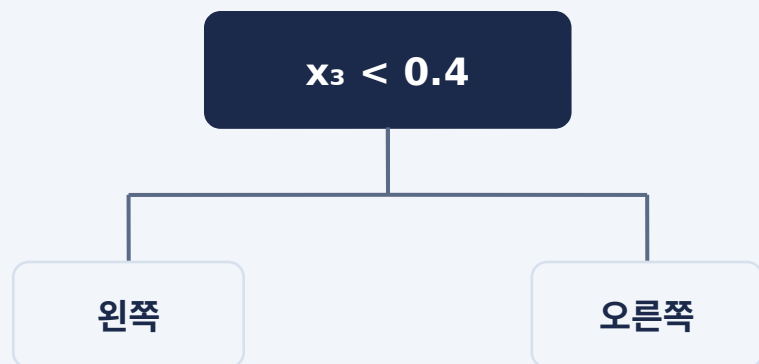
Gain 이 0 이하이면 분할하지 않음 → γ
가 클수록 보수적인 (작은) 트리

min_child_weight

자식 노드의 H 합 (회귀에선 관측치 수)
이 이 값보다 작으면 분할 금지

결측치 처리와 빠른 분할 탐색

결측치 : “ 기본 방향 ” 을 학습한다



x_3 결측 → 왼쪽 (학습된 기본 방향)

분할마다 결측치를 왼쪽 / 오른쪽에 모두 보내 보고 Gain 이 더 큰 쪽을 기본 방향으로 저장 . 결측치를 미리 채울 필요가 없다 (sparsity-aware split)

빠른 분할 탐색 (tree_method)

exact

모든 분할점 검사 . 정확하지만 느림

approx

분위수 (quantile sketch) 로 후보를 줄임

hist (현재 기본값)

연속형 변수를 최대 256 개 구간 (bin) 으로 미리 나눠 히스토그램으로 분할 → LightGBM 과 같은 아이디어

+ 병렬 처리 , GPU(device="cuda"), 대용량 외부 메모리

XGBoost 3.x 문서 기준: tree_method 기본값 "auto" = hist, max_bin 기본 256, grow_policy 기본 depthwise (lossguide 선택 가능)

XGBoost 의 장점과 단점

장점

- 정규화 (γ , λ , α) 로 과대적합 제어가 체계적
- 결측치 자동 처리 , 다양한 손실함수 · 사용자 정의 손실
- 검증된 안정성과 방대한 사용 사례 · 문서
- GPU · 분산 학습 , R·Python·Spark 등 폭넓은 지원

단점 · 주의점

- 튜닝할 하이퍼파라미터가 많다
- 범주형 변수 지원 (enable_categorical) 은 비교적 최근 기능 → 전통적으로는 원 - 핫 인코딩 필요
- level-wise(depthwise) 성장 기본 → 아주 큰 데이터에서는 LightGBM 보다 느린 경우가 많다

기본값 : `eta = 0.3, max_depth = 6, min_child_weight = 1, gamma = 0, lambda = 1, alpha = 0, subsample = 1, colsample_bytree = 1`

PART 2

LightGBM



Light Gradient Boosting Machine — “ 같은 정확도를 , 훨씬 빠르게 ”

LightGBM 의 네 가지 속도 비결

1 히스토그램 분할

연속형 값을 최대 255 개 구간으로 묶어 구간 경계만 분할 후보로

→ 분할 탐색 비용 ↓

2 Leaf-wise 성장

이득이 가장 큰 잎부터 하나씩 분할 (best-first)

→ 같은 잎 수로 더 큰 손실 감소

3 GOSS

기울기가 큰 관측치는 모두, 작은 관측치는 일부만 사용

→ 관측치 수 ↓

4 EFB

동시에 0 이 아닌 적이 없는 희소 변수들을 하나로 묶음

→ 변수 수 ↓

Ke et al. (2017), LightGBM: A Highly Efficient Gradient Boosting Decision Tree. 범주형 변수도 원-핫 없이 직접 처리

히스토그램 기반 분할

고전적 방법 (정렬 기반)

후보 15 개



관측치 n 개 $\rightarrow$ 분할점 후보 $n - 1$ 개 . 노드마다 정렬 · 검사 $\rightarrow$ 데이터가 크면 매우 느림

히스토그램 방법

후보 5 개



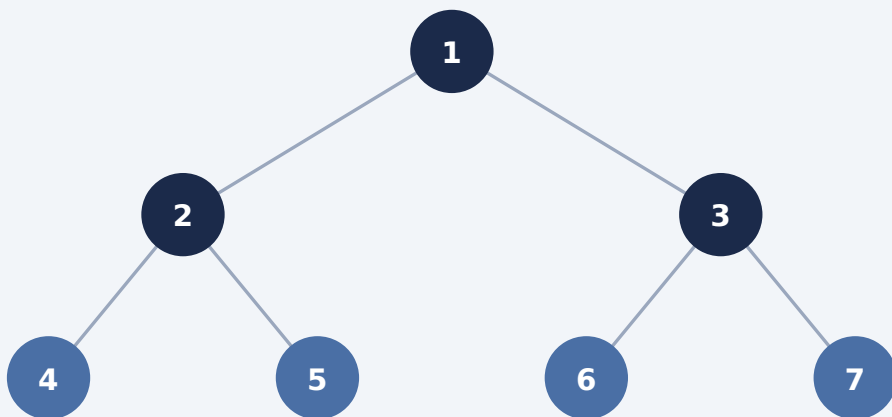
학습 전에 값을 구간 (bin, 최대 255 개) 으로 한 번만 나눈다 .
노드에서는 구간별 G, H 합 (히스토그램) 만 쌓고 , 구간 경계만 검사

+ 히스토그램 뺄셈 : 형제 노드 = 부모 - 다른 자식 $\rightarrow$ 계산 절반

지금은 XGBoost(hist) 도 같은 방법을 기본으로 사용 $\rightarrow$ “히스토그램” 은 현대 boosting 의 공통 표준

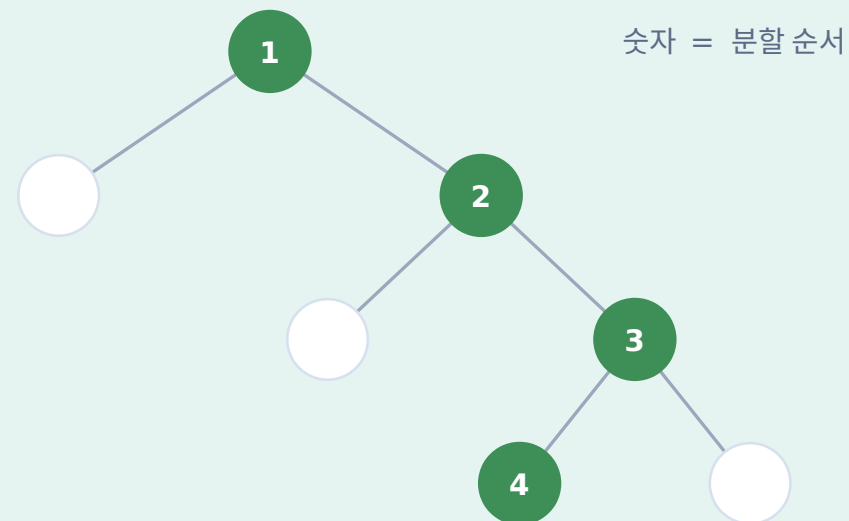
Level-wise vs Leaf-wise 성장

Level-wise (depthwise) · XGBoost 기본



같은 깊이의 노드를 모두 나눈 뒤 다음 깊이로 → 균형 잡힌 트리, 이득이 작은 분할도 수행

Leaf-wise (best-first) · LightGBM 기본

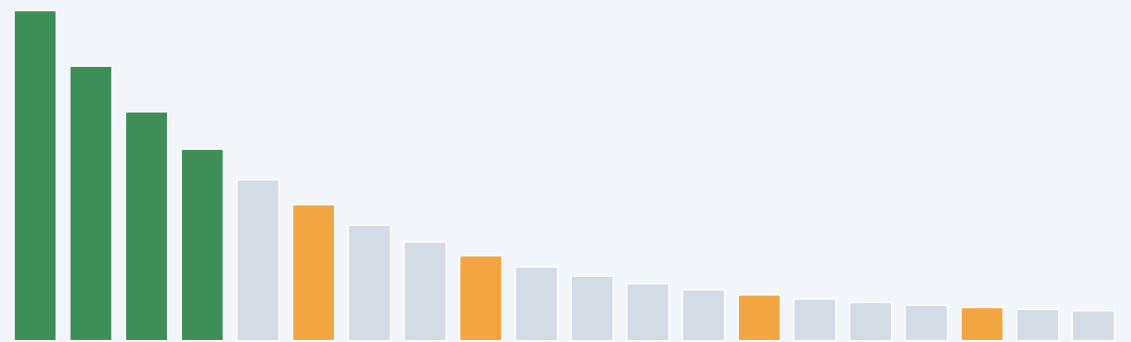


매번 손실 감소가 가장 큰 잎 하나만 분할 → 같은 잎 수에서 더 정확, 대신 한쪽으로 깊어질 수 있음

주의 : leaf-wise 는 작은 데이터에서 과대적합하기 쉽다 → 트리 크기를 깊이가 아니라 num_leaves(기본 31) 로 조절하고 , min_data_in_leaf · max_depth 로 제한

GOSS: 관측치를 줄이고, EFB: 변수를 줄인다

GOSS (Gradient-based One-Side Sampling)



| 기울기 | 큰 순서로 정렬 →

- 상위 $a = 20\%$: 모두 사용 (아직 많이 틀린 관측치)
- 나머지에서 $b = 20\%$ 만 무작위 추출, 가중치 $\times (1-a)/b = 4$
- 나머지는 이번 트리에서 제외

EFB (Exclusive Feature Bundling)

x_A	x_B	묶음
1	0	1
0	3	3+1=4
2	0	2
0	1	1+1=2
0	0	0

x_A 와 x_B 는 동시에 0 이 아닌 적이 없다 (상호배타적) → x_B 에 x_A 의 범위만큼 offset 을 더해 한 변수로 합침

원 - 핫 인코딩처럼 희소한 변수가 수천 개일 때 특히 효과적

둘 다 “정확도는 거의 잃지 않고 계산량만 줄이는” 근사 기법

LightGBM 의 범주형 처리 , 장점과 단점

범주형 변수 직접 처리 : 수준을 기울기 통계 ($\Sigma G / \Sigma H$) 순으로 정렬한 뒤 “정렬된 순서에서의 최적 분할” 을 찾음 $\rightarrow 2^{(k-1)}-1$ 개를 다 보지 않아도 된다 (지난 CART 시간의 정렬 트릭). 수준이 적으면 ($\text{max_cat_to_onehot} = 4$ 이하) 원 - 핫 방식

장점

- 대용량 데이터에서 매우 빠르고 메모리 효율적
- leaf-wise 로 같은 트리 크기에서 높은 정확도
- 범주형 변수 직접 처리 , 결측치 자동 처리
- 분산 · GPU 학습 지원

단점 · 주의점

- 작은 데이터 (수천 행 이하) 에서 과대적합하기 쉬움 $\rightarrow$ `num_leaves`, `min_data_in_leaf` 조절 필수
- `num_leaves` 와 `max_depth` 가 서로 얽혀 튜닝이 헷갈림 (`num_leaves < 2^{max_depth}` 권장)
- 수준이 매우 많은 범주형 변수에서는 과대적합 가능 (`cat_smooth` 등으로 완화)

PART 3

CatBoost

Categorical Boosting — “ 범주형 변수와 target leakage 문제를 정면으로”

CatBoost 의 세 가지 아이디어

1

Ordered Target Statistics

범주형 변수를 “앞선 관측치의 y 만으로”
수치화 → target leakage 없이 인코딩

2

Ordered Boosting

관측치 i 의 잔차를 i 를 보지 않은 모형으로
계산 → 기울기의 편향 (prediction shift)
제거

3

대칭 트리 (Oblivious tree)

같은 깊이의 모든 노드가 같은 분할 조건
사용 → 규칙적이고 과대적합에 강하며
예측이 매우 빠름

공통된 문제의식 : “ 학습에 쓴 y 를 다시 그 관측치의 입력 (인코딩 · 잔차) 에 쓰면 모형이 스스로를 속인다 (target leakage)” →
순서 (permutation) 를 정해 과거 정보만 쓰자

범주형 변수의 수치화 : 왜 조심해야 하나 ?

원 - 핫 인코딩은 수준이 많으면 (예 : 우편번호 수천 개) 변수가 폭발 → 흔한 대안 : 타깃 인코딩 = 범주별 y 의 평균으로 바꾸기

i	범주	y	단순 평균 인코딩
1	A	1	0.75
2	B	0	0.33
3	A	1	0.75
4	A	0	0.75
5	B	1	0.33
6	C	1	1.00
7	A	1	0.75
8	B	0	0.33

문제 : target leakage

범주 C 는 한 번만 등장 → 인코딩 값 1.00 = 자기 자신의 y

모형은 “이 변수가 y 를 완벽히 예측한다” 고 착각 . 학습 데이터에서는 잘 맞지만 새 데이터에서는 무너진다 (과대적합)

수준이 많고 관측치가 적은 범주일수록 심각

Ordered Target Statistics: 앞선 관측치의 y 만 사용

$$TS_i = (\text{앞선 같은 범주의 } y \text{ 합} + a \cdot p) / (\text{앞선 같은 범주의 개수} + a)$$

i (무작위 순서)	범주	y	Ordered TS
1	A	1	0.625
2	B	0	0.625
3	A	1	0.813
4	A	0	0.875
5	B	1	0.313
6	C	1	0.625
7	A	1	0.656
8	B	0	0.542

사전값 p, a

p = 전체 y 평균 (= 5/8), a = 1. 처음 등장하는 범주는 사전값 0.625

범주 C (i = 6)

앞선 C 가 없음 → 0.625. 자기 y 를 쓰지 않으므로 leakage 없음

범주 A

i=1: 0.625 → i=3: (1+0.625)/2 = 0.813 → i=4: 0.875 → i=7: 0.656

여러 순열

순서 효과를 줄이려 여러 무작위 순열을 사용

온라인 학습처럼 “시간 순서대로 도착한 데이터” 라고 가정하고 과거만 보는 것. 예측(test) 시에는 전체 학습 데이터의 통계를 사용

Ordered Boosting: 잔차를 “나를 보지 않은 모형” 으로

기존 boosting 의 미묘한 문제

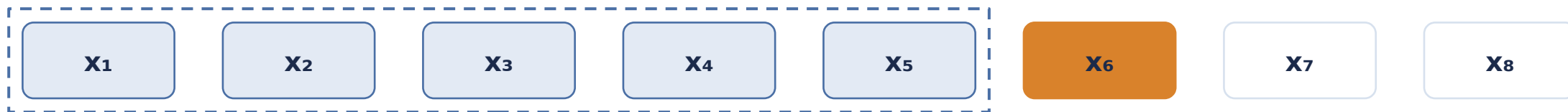
잔차 $r_i = y_i - f(x_i)$ 를 계산하는 모형 f 는 이미 관측치 i 로 학습된 모형 $\rightarrow$ 학습 데이터에서의 잔차는 새 데이터의 잔차보다 체계적으로 작다

$\rightarrow$ 기울기가 편향됨 (prediction shift): 학습 데이터에 대한 일종의 target leakage

CatBoost 의 해결

무작위 순서를 정하고, 관측치 i 의 잔차는 순서상 i 보다 앞선 관측치들만으로 학습한 모형 $M(i-1)$ 로 계산

$\rightarrow$ 잔차가 “처음 보는 데이터에 대한 잔차” 처럼 편향 없이 계산됨

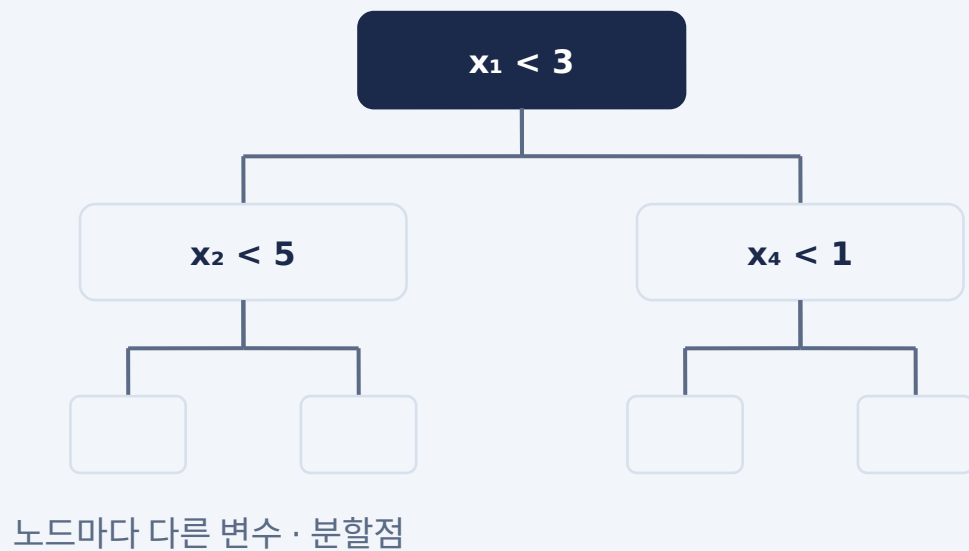


M_5 : $x_1 \sim x_5$ 로만 학습한 모형 $\rightarrow x_6$ 의 잔차 계산

실제로는 계산량을 줄이기 위해 몇 개의 순열과 로그 간격의 부분 모형만 유지. 설정: `boosting_type = "Ordered" / "Plain"` (기본값은 데이터 크기 등에 따라 자동 선택)

대칭 (Oblivious) 트리

일반 트리



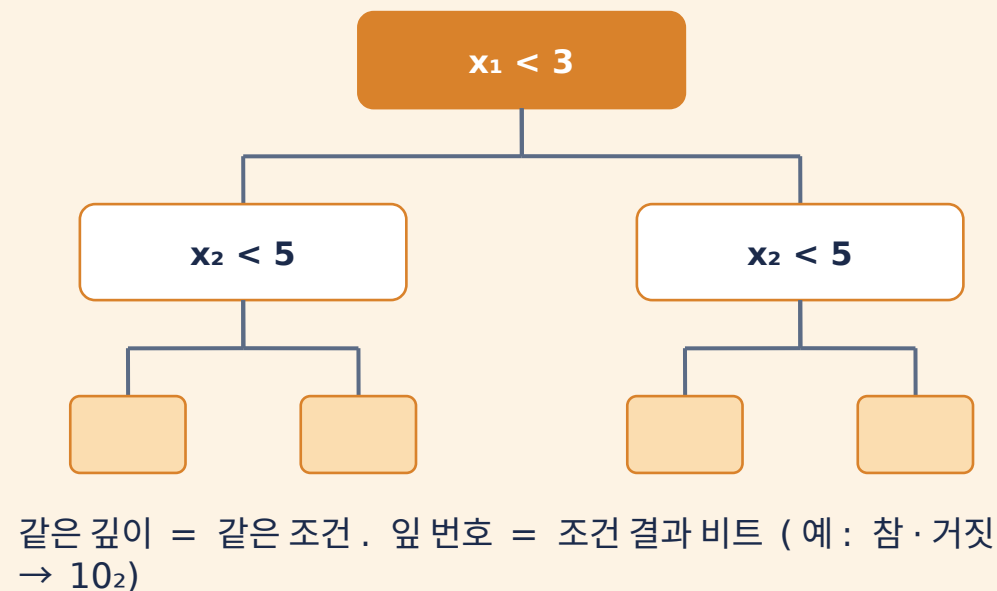
과대적합에 강함

트리 모양이 단순 · 규칙적이라 일종의 정규화 효과

예측이 매우 빠름

d 개의 조건만 평가해 앞 번호를 비트로 계산

대칭 트리 (CatBoost 기본)



대가

유연성이 낮아 같은 깊이에서 표현력은 작을 수 있음
(grow_policy 로 변경 가능)

CatBoost 의 장점과 단점

장점

- 범주형 변수를 전처리 없이 지정만 하면 됨 (cat_features)
- 기본값만으로도 성능이 좋은 편 → 튜닝 부담이 적음
- target leakage·prediction shift 를 줄이는 설계로 과대적합에 강함
- 대칭 트리 덕분에 예측 (서빙) 이 매우 빠름

단점 · 주의점

- CPU 에서 학습 속도는 셋 중 가장 느린 편 (특히 ordered 모드)
- 범주형 변수가 없는 순수 수치 데이터에서는 이점이 줄어들음
- R 패키지가 CRAN 에 없음 → GitHub 릴리스에서 설치
- 메모리 사용량이 큰 편

기본값 : iterations = 1000, depth = 6, l2_leaf_reg = 3, learning_rate = 자동 설정, grow_policy = SymmetricTree

PART 4

비교와 선택



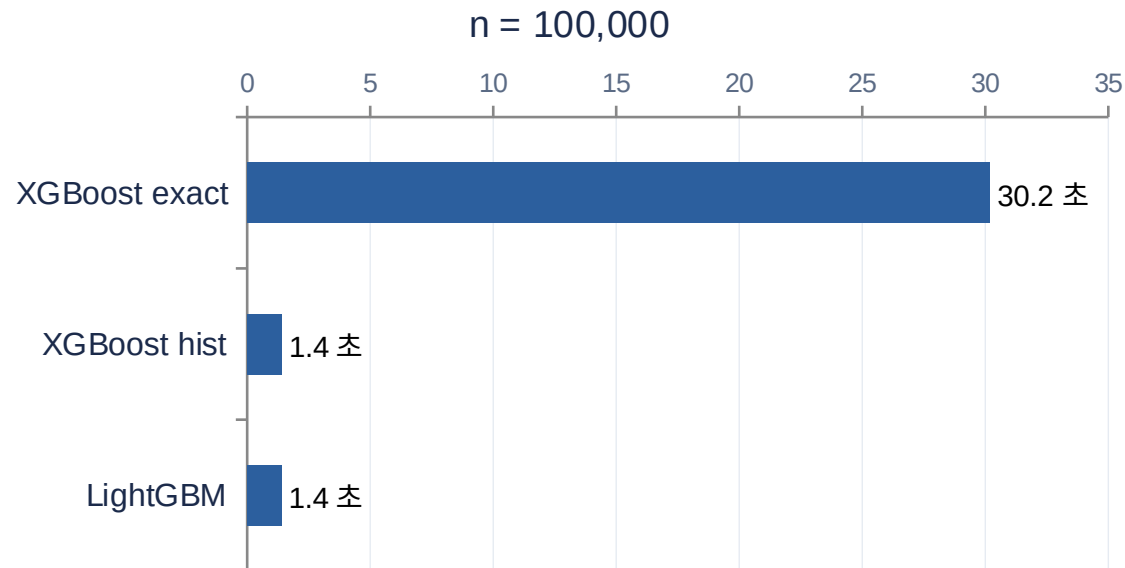
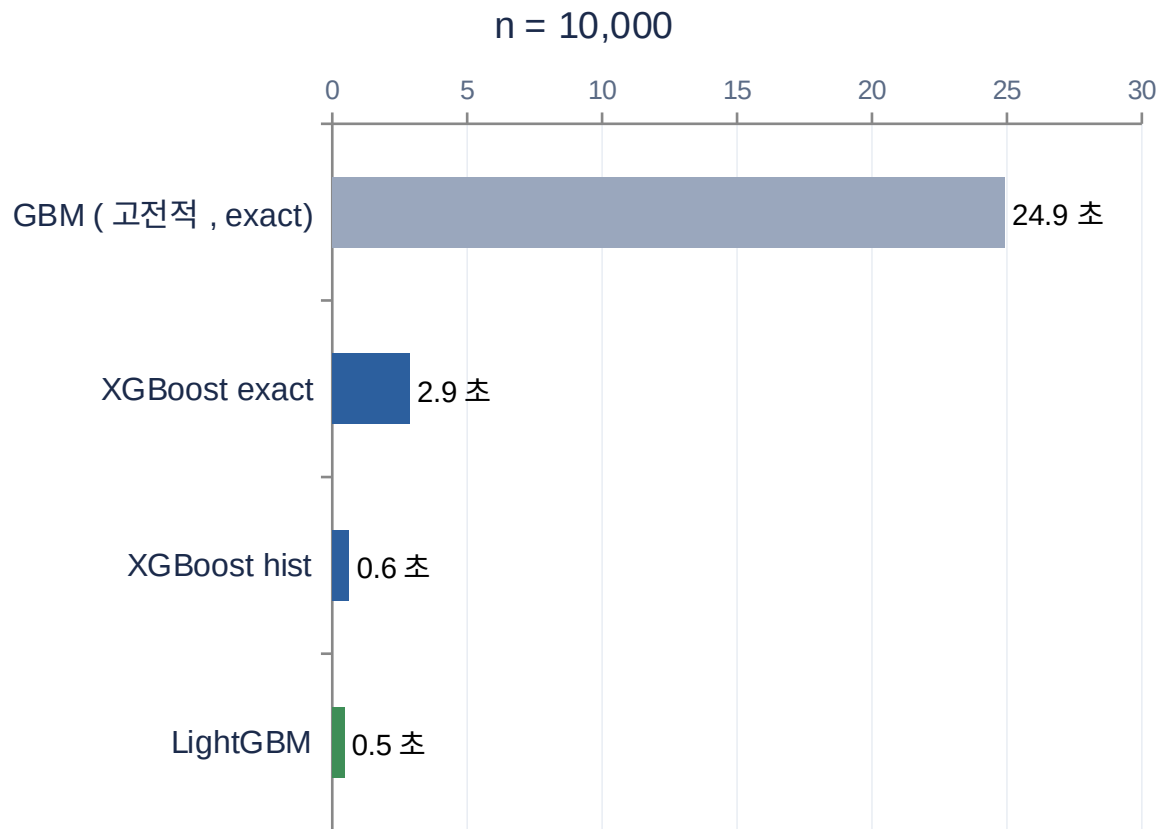
세 모형은 무엇이 같고 , 무엇이 다른가

한눈에 비교

	XGBoost	LightGBM	CatBoost
핵심 아이디어	정규화 목적함수 + 2 차 근사	히스토그램 + leaf-wise + GOSS·EFB	Ordered TS + ordered boosting
트리 성장	level-wise (기본)	leaf-wise	대칭 (oblivious) 트리
분할 탐색	hist (기본) / exact / approx	히스토그램	히스토그램 (border)
범주형 변수	지원 (enable_categorical)	직접 지원 (정렬 분할)	가장 강함 (ordered TS)
결측치	기본 방향 학습	자동 처리	자동 처리 (Min 방향)
학습 속도	빠름	가장 빠른 편 (대용량)	느린 편 (CPU)
기본값 성능	보통	보통 (작은 데이터 주의)	좋은 편
트리 크기 조절	max_depth	num_leaves	depth
R 설치	CRAN	CRAN	GitHub 릴리스

속도·성능은 데이터와 설정에 따라 크게 달라진다. 표는 일반적인 경향

히스토그램 방법은 얼마나 빠를까 ? (직접 측정)

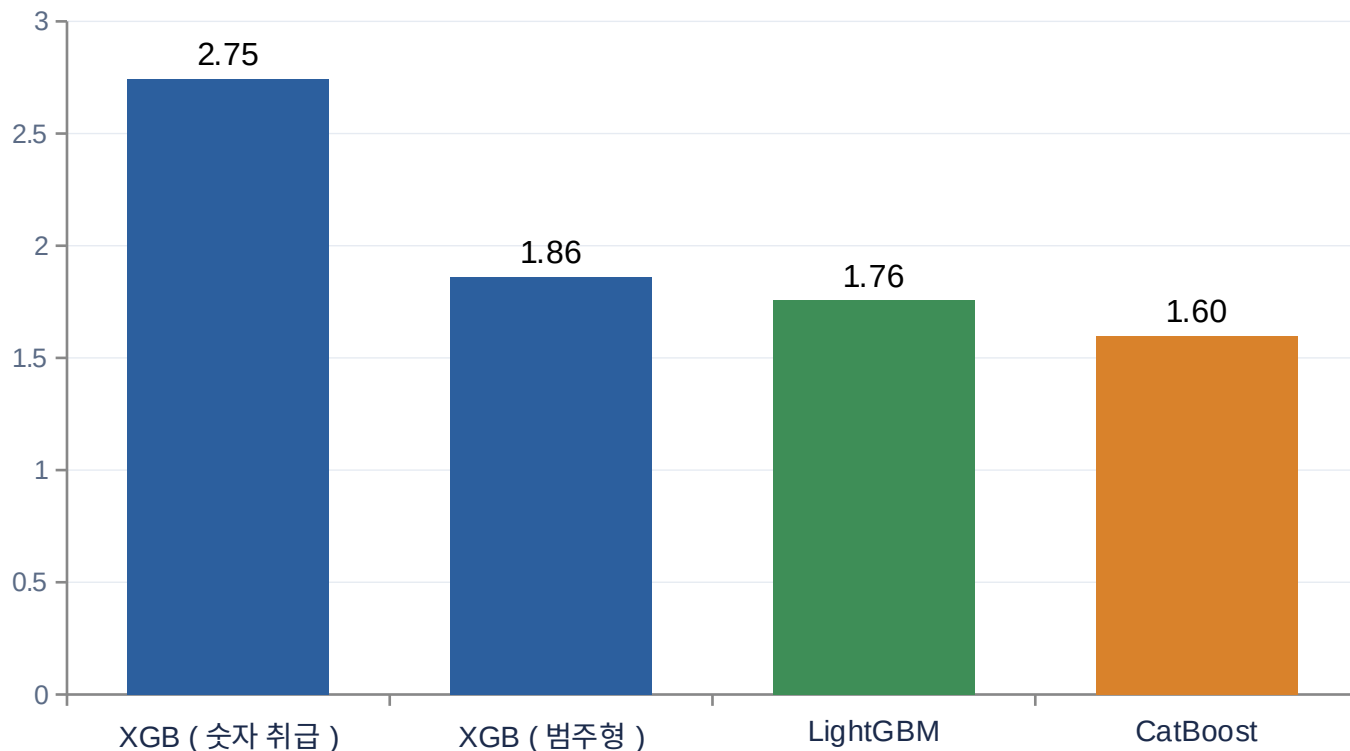


고전적 GBM(정렬 기반)은 10,000 행에서 약 25 초 → 히스토그램 방식은 0.5 초 내외 . 데이터가 10 배 커지면 exact 는 30 초 , hist·LightGBM 은 1.4 초

트리 200 개, 깊이 6(LightGBM 은 num_leaves 63), 변수 20 개, CPU 2 코어. 한 번 측정한 값으로, 환경에 따라 달라짐. 고전적 GBM = scikit-learn GradientBoostingRegressor

범주형 변수가 있는 데이터에서의 예측 정확도 (직접 실험)

테스트 RMSE (작을수록 좋음 , 잡음 하한 = 1.5)



범주를 숫자로 취급하면

XGBoost 에 범주 코드 (0~499) 를 그대로 넣으면 RMSE 2.75 — 범주 사이의 “순서” 라는 가짜 정보를 학습

범주형으로 처리하면

XGBoost native 1.86, LightGBM 1.76, CatBoost 1.60 → 범주형 처리 방식이 성능을 좌우

대가 : 시간

CatBoost 33 초 (3000 회 모두 사용) vs 다른 모형 2~4 초

모의 데이터: $n = 40,000$ (검증 10,000, 테스트 20,000), 수치형 8 개 + 범주형 2 개 (수준 500 개, 20 개, 범주별 효과 존재). 학습률 0.05, 깊이 6 (LightGBM num_leaves 31), 조기 종료. 한 번의 실험일 뿐 일반적 순위가 아님

언제 무엇을 쓸까 ? — 실용적인 출발점

XGBoost

- 가장 무난한 기본 선택, 자료·예제가 가장 많음
- 정규화 옵션을 세밀하게 조절하고 싶을 때
- 사용자 정의 손실함수, 다양한 환경 (Spark 등)

LightGBM

- 행이 수십만 ~ 수백만인 대용량 데이터
- 빠르게 여러 번 실험해야 할 때 (튜닝 반복)
- 희소한 변수가 매우 많을 때 (EFB)

CatBoost

- 범주형 변수가 많고 수준도 많을 때
- 튜닝 시간이 부족해 기본값으로 좋은 성능이 필요할 때
- 예측 (서빙) 속도가 중요할 때

가장 확실한 방법 : 셋 다 적합해 같은 CV 로 비교하기 . 차이는 보통 “모형 선택” 보다 “튜닝과 변수 처리” 에서 더 크게 난다

R 실습 : 세 라이브러리 기본 사용법 (조류 데이터 a1)

R CONSOLE

```
# XGBoost
> library(xgboost)
> dtr <- xgb.DMatrix(data.matrix(train[,1:11]), label=train$a1)
> dva <- xgb.DMatrix(data.matrix(valid[,1:11]), label=valid$a1)
> xg <- xgb.train(params=list(objective="reg:squarederror", eta=0.05, max_depth=4),
+ data=dtr, nrounds=3000, evals=list(valid=dva), early_stopping_rounds=100)
# LightGBM
> library(lightgbm)
> ltr <- lgb.Dataset(data.matrix(train[,1:11]), label=train$a1)
> lg <- lgb.train(params=list(objective="regression", learning_rate=0.05, num_leaves=15),
+ data=ltr, nrounds=3000, valids=list(valid=lgb.Dataset.create.valid(ltr,
+ data.matrix(valid[,1:11]), label=valid$a1)), early_stopping_rounds=100)
# CatBoost ( 범주형 : season, size, speed = 1~3 번째 열 )
> library(catboost)
> ptr <- catboost.load_pool(train[,1:11], label=train$a1)
> cbm <- catboost.train(ptr, catboost.load_pool(valid[,1:11], label=valid$a1),
+ params=list(loss_function="RMSE", iterations=3000, learning_rate=0.05, od_type="Iter", od_wait=100))
```

CatBoost: factor 열을 범주형으로 자동 인식. 설치는 `remotes::install_url(GitHub 릴리스 주소)` — catboost.ai 의 R 설치 문서 참고

SUMMARY

오늘의 핵심 정리

1 같은 뼈대

셋 다 “잔차 (음의 기울기) 를 작은 트리로 반복 적합” 하는 gradient boosting

2 XGBoost

정규화 목적함수 ($\gamma T + \frac{1}{2}\lambda \sum w^2$) + 2 차 근사 ($w_j^* = -G/(H+\lambda)$) + 결측 기본 방향

3 LightGBM

히스토그램 + leaf-wise 성장 + GOSS·EFB → 대용량에서 빠름 , 작은 데이터에선 과대적합 주의

4 CatBoost

Ordered TS · ordered boosting 으로 target leakage 차단 + 대칭 트리 . 범주형 변수에 강함

다음 : Boosting 모형 튜닝 — 세 라이브러리 공통의 튜닝 전략